

NLP PT1 Question Bank - Descriptive Answers

This document provides structured, exam-oriented answers for the descriptive questions (1 to 18) of the NLP PT1 Syllabus.

1. What is Natural Language Processing? Explain generic NLP system?

Definition of NLP

Natural Language Processing (NLP) is a subfield of computer science, artificial intelligence, and computational linguistics concerned with the interactions between computers and human (natural) languages. The goal is to enable computers to understand, interpret, generate, and manipulate human language in a useful way.

Generic NLP System Architecture

A generic NLP system processes natural language text input through a series of pipeline stages to understand or generate meaning.



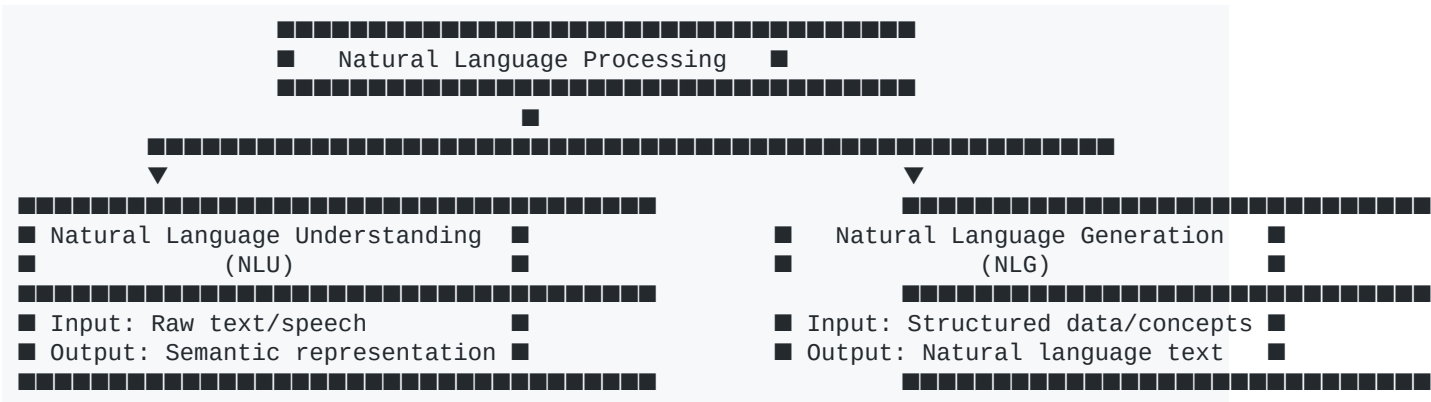
Components of a Generic NLP System

- **Input Interface:** Accepts text or speech input.

- **Knowledge Base / Lexicon:** A repository of words, grammar rules, and semantic relations.
- **Processing Pipeline:**
 1. **Parser/Analyzer:** Converts raw input into structured representations.
 2. **Disambiguation Engine:** Resolves syntactic, semantic, and lexical ambiguities.
- **Output Generator (for NLG):** Generates natural-sounding response text.

2. What are the components of NLP?

NLP is broadly divided into two major components: **Natural Language Understanding (NLU)** and **Natural Language Generation (NLG)**.



A. Natural Language Understanding (NLU)

NLU is the process of mapping natural language input onto a structured representation of its meaning. It involves:

- **Lexical Analysis:** Mapping words to their dictionary meanings and parts of speech.
- **Syntactic Analysis:** Analyzing words in a sentence to uncover the grammatical structure.
- **Semantic Analysis:** Determining the literal meaning of sentences.
- **Pragmatic Analysis:** Interpreting the meaning of language in context.

B. Natural Language Generation (NLG)

NLG is the process of producing understandable text in human language from a representation of knowledge. It involves three main stages: 1. **Document Planning:** Deciding what content to include and how to structure it. 2. **Microplanning:** Choosing appropriate words (lexical selection), referring expressions, and aggregation. 3. **Realization:** Applying grammatical rules to generate the final surface text.

3. Explain about the phases of Natural Language Processing?

NLP systems process language through a hierarchical series of levels or phases.

Phase	Description	Example
-------	-------------	---------

1. Phonological Phase	Analysis of speech sounds and pronunciation rules (speech-to-text systems only).	Map sound waves to phonemes /kæt/ → "cat" .
2. Morphological Phase	Analyzing the structure of individual words and how they are constructed from morphemes.	"unhappiness" → un- (prefix) + happy (root) + -ness (suffix).
3. Lexical Phase	Determining the parts of speech (POS) and dictionary meaning of individual words.	"book" can be a Noun (read a book) or a Verb (book a ticket).
4. Syntactic Phase	Analyzing sentences to verify grammatical structure and determine word relations.	"The cat sat on the mat" → Subject: "The cat" , Verb: "sat" , PrepPhrase: "on the mat" .
5. Semantic Phase	Extracting the literal meaning of sentences and resolving word sense ambiguity.	Disambiguating "bank" in "river bank" vs "money bank" .
6. Discourse Phase	Analyzing relationships between consecutive sentences (coreference resolution).	"John sold his car. He bought a bike." → "He" refers to "John" .
7. Pragmatic Phase	Interpreting the meaning based on situational context, intent, and real-world knowledge.	"Can you pass the salt?" → Interpreted as a request, not a question about physical ability.

4. Explain challenges and applications of NLP?

Challenges in NLP

- **Ambiguity:** Natural language is highly ambiguous at lexical, syntactic, semantic, and pragmatic levels.
- **Out of Vocabulary (OOV) / Neologisms:** New words, slang, and emojis emerge constantly (e.g., **"ghosting"**, **"lowkey"**).
- **Sarcasm and Irony:** Literal meaning is opposite to the intended sentiment.
- **Context Dependency:** Meaning changes completely based on situational context or domain.
- **Low-Resource Languages:** Lack of large corpora and annotated datasets for regional/tribal languages.

Applications of NLP

- **Machine Translation (MT):** Translating text/speech from one language to another (e.g., Google Translate).
- **Sentiment Analysis:** Extracting subjective opinion and emotion from text (e.g., analyzing product reviews).
- **Information Retrieval and Extraction:** Searching documents and extracting structured facts.
- **Virtual Assistants & Chatbots:** Conversational agents (e.g., Siri, Alexa, customer service bots).
- **Text Summarization:** Condensing long documents into short, informative summaries.
- **Spam Detection & Content Moderation:** Automatically filtering emails or flagging abusive posts.

5. Explain types of ambiguities and ways to resolve it with examples?

Types of Ambiguity

1. Lexical Ambiguity

Occurs when a single word has multiple meanings.

- **Example:** *"She went to the bank."* (River bank or financial bank?)
- **Resolution:** **Word Sense Disambiguation (WSD)** using contextual clues or WordNet.

2. Syntactic / Structural Ambiguity

Occurs when a sentence can be parsed in multiple grammatical ways.

- **Example:** *"I saw the man with the telescope."* (Did I use the telescope, or does the man have a telescope?)
- **Resolution:** Grammatical parsing tree evaluation, semantic constraints, and probabilistic parsers.

3. Semantic Ambiguity

Occurs when the literal meaning of the sentence is ambiguous, even when syntax is resolved.

- **Example:** *"All students love one teacher."* (Do they all love the *same* teacher, or does each student have *a* teacher they love?)
- **Resolution:** Quantifier scope resolution and logical forms representation.

4. Anaphoric / Pronoun Ambiguity

Occurs when it is unclear what a pronoun refers to.

- **Example:** *"The trophy did not fit into the suitcase because it was too large."* (What was too large? The trophy or the suitcase?)
- **Resolution:** **Coreference Resolution** using semantic consistency, Winograd schema models.

5. Pragmatic Ambiguity

Occurs when the intention of the speaker is unclear without context.

- **Example:** *"It's hot in here."* (Is this a simple statement, or a request to turn on the AC?)
- **Resolution:** Context models, dialog state tracking, and intent classification.

6. What is the output of Morphological Analysis for Regular Verb, Irregular Verb, Singular noun, Plural noun?

Morphological analysis breaks words down into their component stems/lemmas and grammatical features.

Word Category	Example Word	Morphological Breakdown (Lexical Entry)	Output Representation
Regular Verb	walks	walk (root) + -s (3rd person singular present)	walk + V + Pres + 3Sg
Irregular Verb	went	go (root, irregular past tense)	go + V + Past

Singular Noun	cat	cat (root noun)	cat + N + Sg
Plural Noun	oxen	ox (root) + -en (irregular plural suffix)	ox + N + Pl

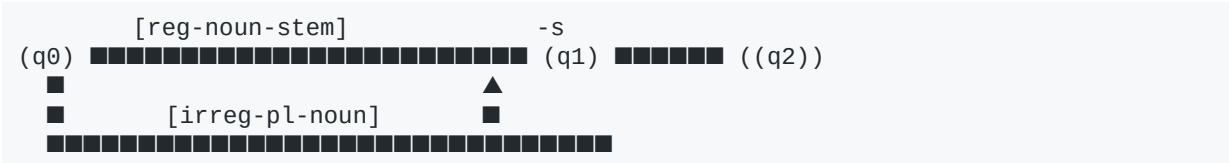
7. What is the significance of FSA in morphological analysis?

A **Finite State Automaton (FSA)** is a mathematical abstraction representing a set of states and transitions. In morphological analysis, FSAs are highly significant because they represent **morphotactics** (the rules governing how morphemes can be combined to form valid words).

Key Significance points:

- 1. **Lexicon Representation:** FSAs can represent dictionaries compactly by sharing paths for common prefixes and suffixes.
- 2. **Word Recognition:** An FSA determines if a given sequence of characters forms a grammatically valid word.
- 3. **Low Memory Footprint:** Minimizing an FSA produces a very efficient data structure for searching.

Morphological FSA Example (Noun Pluralization):



- **q0** to **q1** matches singular nouns like **"cat"**.
- **q1** to **q2** matches plurals ending in ***-s*** (**"cats"**).
- **q0** to **q2** directly matches irregular plurals like **"geese"**.

8. Explain Role of FST in Morphological Parsing with an example?

A **Finite State Transducer (FST)** is a type of finite state automaton that has **two tapes**: an input tape and an output tape. It maps an input string to an output string.

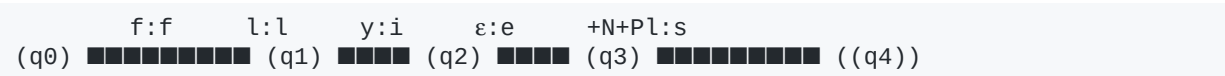
Role of FST:

- FSTs are used for bidirectional mapping:
- **Morphological Parsing (Analysis):** Maps a surface form to its lexical form (e.g., **flies** → **fly + N + Pl**).
- **Morphological Generation:** Maps a lexical form to a surface form (e.g., **fly + N + Pl** → **flies**).
- It handles spelling rules (orthographic rules) and morpheme boundaries.

Example: Parsing "flies"

Surface level: **f l i e s** Lexical level: **f l y + N + Pl**

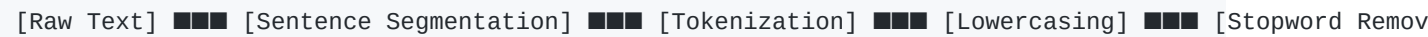
An FST implements this mapping by using transitions labeled with **input:output** symbols:



*(Here *epsilon* represents the empty string, modeling the insertion of 'e' during the y → ie spelling change rule).*

9. Explain pre-processing steps generally used in NLP.

Pre-processing prepares raw text for analysis by cleaning it and normalizing its format.



Steps:

- 1. Sentence Segmentation (Splitting):** Breaking a large text corpus into individual sentences using delimiters like **., !, ?**.
- 2. Tokenization:** Segmenting running text into individual words, punctuation marks, and symbols.
 - *Example:* **"Dr. Smith's cat"** → **["Dr.", "Smith", "'s", "cat"]**.
- 3. Lowercasing:** Converting all text to lowercase to ensure consistency (e.g., **"Cat"** → **"cat"**).
- 4. Stopword Removal:** Eliminating highly frequent words that contain little semantic value (e.g., **"the"**, **"is"**, **"at"**, **"which"**).
- 5. Punctuation and Special Character Removal:** Filtering out symbols that are not useful for downstream tasks (except where punctuation matters, like POS tagging).
- 6. Normalization (Stemming/Lemmatization):** Reducing inflected words to their root forms.

10. Compare Stemming and Lemmatization?

Both stemming and lemmatization reduce words to their base form, but they do so differently.

Feature	Stemming	Lemmatization
Method	Rule-based heuristics that chop off prefixes/suffixes.	Uses lexical databases, POS tags, and vocabulary search.
Output	May result in non-dictionary words (stems).	Always returns a valid dictionary word (lemma).
Speed	Fast (simple string operations).	Slower (requires dictionary lookup and POS tagging).
Accuracy	Lower (prone to over-stemming and under-stemming).	Higher (context-aware).
Context Aware	No.	Yes.
Example: "better"	better	good (adjective)
Example: "saw"	saw	see (verb) or saw (noun)
Example: "studies"	studi	study

11. Explain Porter Stemmer algorithm in detail.

The **Porter Stemmer** is a lexicon-free suffix-stripping algorithm designed by Martin Porter in 1980. It uses a set of cascading rules grouped into **5 sequential phases**.

Definition of Word Structure:

A word in the Porter Stemmer is modeled as:

[C] (VC)^m [V] Where:

- **C** is a sequence of consonants.
- **V** is a sequence of vowels.
- **m** is the **Measure** of the word (representing count of VC pairs).

The Five Phases:

1. **Phase 1:** Handles plurals and third-person singular inflections.

- *Rules:* **sses** → **ss** (*caresses* → *caress*); **ies** → **i** (*ponies* → *poni*); **ss** → **ss**; **s** → *lepsilon*.

2. **Phase 2:** Simplifies complex suffixes when the measure $m > 0$.

- *Rules:* **eed** → **ee** (*agreed* → *agree*); **ing** → *lepsilon* (*walking* → *walk*); **ed** → *lepsilon*.

3. **Phase 3:** Converts **y** to **i** if there is another vowel in the stem.

- *Rule:* ***Y** → **I** (*happy* → *happi*).

4. **Phase 4:** Simplifies double suffixes (e.g., derivational morphology).

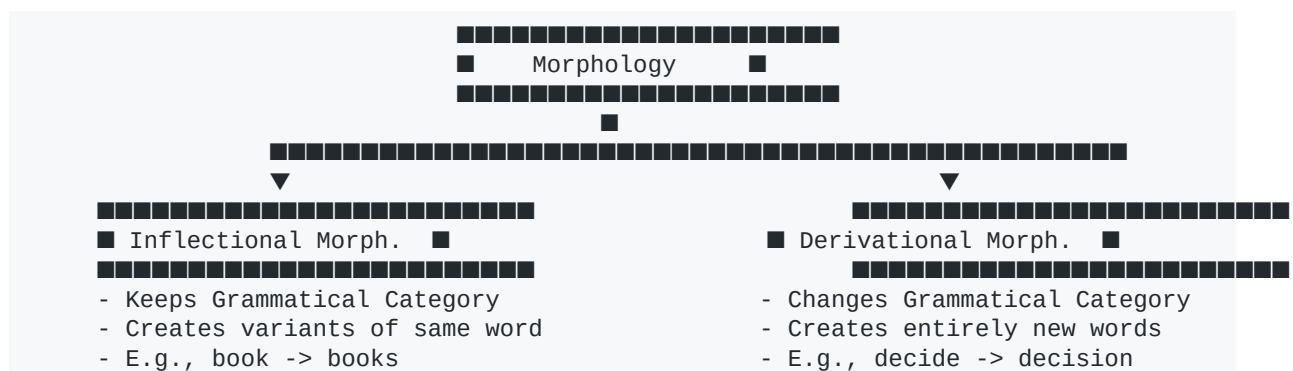
- *Rules:* **ational** → **ate** (*relational* → *relate*); **izer** → **ize** (*digitizer* → *digitize*); **tional** → **tion**.

5. **Phase 5:** Handles cleanup steps, such as removing final **e** or double consonants.

- *Rule:* **probate** → **probat** (if $m > 1$); **controlling** → **control**.

12. Illustrate Inflectional and Derivational morphology with an example?

Morphology is the study of how words are structured. It is divided into inflectional and derivational.



Inflectional Morphology

The process of adding grammatical suffixes to a word without changing its core meaning or grammatical category (Part of Speech).

- **Purpose:** Marks properties like tense, number, gender, case, or person.
- **Example:**
 - **cat** (Noun) + **-s** → **cats** (Noun) — *Plurality*
 - **walk** (Verb) + **-ed** → **walked** (Verb) — *Past tense*

Derivational Morphology

The process of forming new words, often changing the grammatical category (Part of Speech) or significantly altering the semantic meaning.

- **Purpose:** Creates a new lexeme with new grammatical behavior.
 - **Example:**
 - **happy** (Adjective) + **-ness** (Suffix) → **happiness** (Noun) — *Category change*
 - **write** (Verb) + **-er** → **writer** (Noun) — *Category change*
 - **lock** (Verb) + **un-** (Prefix) → **unlock** (Verb) — *Meaning alteration (Reversal)*
-

13. Explain Affixes and types of Affixes.

An **Affix** is a bound morpheme that attaches to a root or stem to modify its meaning or grammatical function.

Types of Affixes based on Position:

1. **Prefix:** Attached to the beginning of a stem.

- *Example:* **un-** in **untidy**, **re-** in **rewrite**.

2. **Suffix:** Attached to the end of a stem.

- *Example:* **-ful** in **beautiful**, **-ing** in **running**.

3. **Infix:** Inserted inside the root word (rare in English, common in languages like Tagalog).

- *Example (Colloquial English):* **"abso-bloody-lutely"**.

4. **Circumfix:** Consists of two parts, one attached at the beginning and one at the end of the stem (common in German).

- *Example (German past participle):* **ge-** + **seh** + **-en** → **gesehen** (seen).
-

14. What is a regular expression? What is significance in word-level analysis?

Definition

A **Regular Expression (Regex)** is a sequence of characters that defines a search pattern. It is used in pattern matching algorithms to find, extract, or replace substrings.

Significance in Word-Level Analysis:

- **Tokenization:** Defining token boundaries (e.g., splitting by spaces `\s+` or non-alphanumeric characters `\W+`).
 - **Validation:** Checking structural formats of tokens (e.g., verifying phone numbers, emails, or zip codes).
 - **Morphological Pattern Matching:** Identifying prefixes or suffixes (e.g., pattern `\w+(ing|ed|s)$` to capture potential inflections).
 - **Text Cleaning:** Stripping out HTML tags, digits, or punctuation.
-

15. Difference between free and bound morpheme?

Feature	Free Morpheme	Bound Morpheme
Definition	A morpheme that can stand alone as a complete word.	A morpheme that cannot stand alone; it must attach to another.
Meaning	Carries a clear lexical or functional meaning on its own.	Serves to modify meaning or add grammatical features.
Examples	dog, run, happy, book, over, the	un-, re-, -s, -ing, -ly, -est
Types	Lexical (content) and Functional (grammatical).	Derivational and Inflectional affixes.

16. Explain following Terms:

Free and Bound Morpheme

- ***Free Morpheme:** The base/root of a word that carries semantic weight on its own (e.g., `*"nation"`).
- ***Bound Morpheme:** Grammatical affixes that require attachment to a root to be meaningful (e.g., `*"-al"` in `*"national"`).

Morphotactics

- **Definition:** The rules that dictate the permissible order of morpheme combinations in a language.
- ***Example:** In English, we can attach the prefix `un-` to the adjective `happy` to make `unhappy`, and then attach `-ness` to make `unhappiness`. However, we cannot combine them in the order `happynessun`.

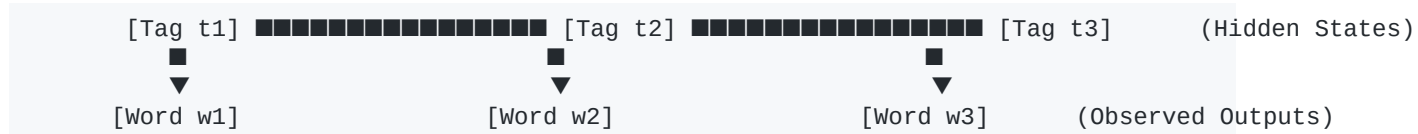
Orthographic Rule

- **Definition:** Rules describing how spelling changes when morphemes are combined.
 - ***Example (Consonant doubling):** `run + -ing → running` (double 'n').
 - ***Example (y-to-i change):** `study + -ed → studied`.
-

17. Explain Hidden Markov Model for POS based tagging? What are the limitations of the HMM model?

HMM for POS Tagging

The **Hidden Markov Model (HMM)** is a generative probabilistic model used to assign the most likely sequence of POS tags to a sequence of words.



In POS tagging:

- The **words** are the **observed states** ($W = w_1, w_2, \dots, w_n$).
- The **POS tags** are the **hidden states** ($T = t_1, t_2, \dots, t_n$).

An HMM uses two sets of probabilities: 1. **Transition Probability (A)**: The probability of moving from tag t_{i-1} to tag t_i . $P(t_i | t_{i-1}) = \frac{\text{Count}(t_{i-1}, t_i)}{\text{Count}(t_{i-1})}$ 2. **Emission Probability (B)**: The probability of tag t_i generating word w_i . $P(w_i | t_i) = \frac{\text{Count}(t_i, w_i)}{\text{Count}(t_i)}$

Using Bayes' theorem and the Markov assumption, the Viterbi algorithm finds the tag sequence T^* that maximizes:

$$T^* = \operatorname{argmax}_T \prod_{i=1}^n P(t_i | t_{i-1}) \times P(w_i | t_i)$$

Limitations of HMM for POS Tagging:

- **Markov Assumption**: It assumes the next tag depends only on the current tag (bigram) or past two tags (trigram), ignoring long-range syntactic dependencies.
- **Independence Assumption**: Word emission depends only on the tag, ignoring context or nearby words.
- **Out of Vocabulary (OOV) Handling**: Difficult to assign correct tags to unseen words because emission probability $P(\text{unseen_word} | t_i)$ is 0.

18. Explain n-gram language model and its application?

Definition

An **n-gram** is a contiguous sequence of n items (words/characters) from a given sample of text. An **n-gram language model** estimates the probability of the next word given the previous $n-1$ words.

- **Unigram ($n=1$)**: Probability of a word occurs independently: $P(w_1, w_2) = P(w_1)P(w_2)$.
- **Bigram ($n=2$)**: Approximates word probability based on the preceding word: $P(w_i | w_{i-1}) = \frac{C(w_{i-1}, w_i)}{C(w_{i-1})}$.
- **Trigram ($n=3$)**: Uses the preceding two words: $P(w_i | w_{i-2}, w_{i-1}) = \frac{C(w_{i-2}, w_{i-1}, w_i)}{C(w_{i-2}, w_{i-1})}$.

Applications of n-grams:

1. **Predictive Text & Autocomplete:** Suggesting the next word in search engines or keyboards. 2. **Speech Recognition:** Disambiguating homophones (e.g., *"two"* vs *"to"* vs *"too"* based on context). 3. **Spell Checking:** Finding words that are unlikely to appear in a sequence. 4. **Machine Translation:** Evaluating text naturalness and fluid phrasing.